第一轮完整测试结果汇总（按 `test_metric_for_4RQs.txt` 口径）

> 说明：
>
> 1. **表组 A**：与 PMD / SpotBugs 的比较
> 2. **表组 B**：模型比较（效果 + 成本），同时回答 RQ2 与 RQ3
> 3. **表组 C**：模块消融实验
>
> bug-unit口径：每一个具体的关键资源对象，都是一个独立缺陷单位 case-level口径：把一个测试样例文件中的 bad 方法看作一个整体缺陷单位 其中：
>
> - **Juliet**：
> - PMD / SpotBugs 使用严格的 **case-level** 口径（bad case / good case）
> - JLeakDetector(当前工具) 自身更适合用 bug-unit 口径评估，但在与 PMD / SpotBugs 横向比较时，这里统一给出 case-level 指标
> - **真实项目**：
> - 按 `test_metric_for_4RQs.txt` 的要求，不再使用 FP / TN / F1 作为主指标
> - 主要关注 `TP_known / FN / recall_known / issue_count / inspection_burden / 时间 / token`

表组 A：与 PMD / SpotBugs 的比较

表 A1：Juliet 上 PMD、SpotBugs 与 JLeakDetector 最好/最坏结果的严格比较

方法	TP	FP	FN	TN	Precision	Recall	F1
PMD	8	8	29	88	0.500	0.216	0.302
SpotBugs	6	0	31	96	1.000	0.162	0.279
JLeakDetector-best	36	1	1	95	0.973	0.973	0.973
JLeakDetector-worst	36	20	1	76	0.643	0.973	0.774

结论（A1）

- 即使在最差运行结果下，JLeakDetector 的 F1 仍明显高于 PMD 和 SpotBugs。
- PMD 的主要问题是 **FP 较多**，容易把 good case 也报出来。
- SpotBugs 的主要问题是 **Recall 偏低**，虽然 Precision 很高，但覆盖能力不足。
- JLeakDetector 的波动主要体现为 **FP 数量变化**，而不是召回崩塌。

表 A2：FitNesse 上 PMD、SpotBugs 与 JLeakDetector 最好/最坏结果的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden
PMD	3	2	0.6000	51.00	10.20
SpotBugs	5	0	1.0000	10.00	2.00
JLeakDetector-best	5	0	1.0000	67.33	12.47

方法	TP_known	FN	recall_known	候选结果数	inspection_burden
JLeakDetector-worst	2	3	0.4000	60.00	11.60

结论（A2）

- 在 FitNesse 上，SpotBugs 与 JLeakDetector 最佳配置都能达到 **1.0 recall_known**。
- 但 JLeakDetector 在该项目上的波动较大，最差情况下会下降到 **0.4**，说明其对流程配置非常敏感。
- SpotBugs 在 FitNesse 上表现异常突出，而且人工审核负担最低。
- PMD 在 FitNesse 上覆盖能力偏弱，仅命中 3 / 5 个已知缺陷。

表 A3：TripleA 上 PMD、SpotBugs 与 JLeakDetector 最好/最坏结果的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden
PMD	12	2	0.8571	124.00	8.86
SpotBugs	7	7	0.5000	26.00	1.86
JLeakDetector-best	14	0	1.0000	90.00	5.14
JLeakDetector-worst	12	2	0.8571	88.33	5.21

结论（A3）

- 在 TripleA 上，JLeakDetector 最佳结果明显优于 PMD 和 SpotBugs。
- JLeakDetector worst退化后大约与 PMD 持平，但仍显著优于 SpotBugs。
- PMD 覆盖较高，但候选报告数过多，人工审核负担较重。
- SpotBugs 候选较少，但已知缺陷覆盖明显不足。

表组 B：模型比较（效果 + 成本）

表 B1：Juliet 上 PMD、SpotBugs、qwen-plus、deepseek-chat、doubao 的比较

方法	TP	FP	FN	Precision	Recall	F1
PMD	8.00	8.00	29.00	0.500	0.216	0.302
SpotBugs	6.00	0.00	31.00	1.000	0.162	0.279
deepseek-chat	35.00	3.67	2.00	0.906	0.946	0.925
qwen-plus	34.75	3.92	2.25	0.901	0.939	0.919
doubao-seed-2-0-mini-260215	35.25	11.83	1.75	0.754	0.953	0.840

结论（B1）

- 三个 JLeakDetector 模型都显著优于 PMD 和 SpotBugs。
- `deepseek-chat` 与 `qwen-plus` 的综合效果最均衡。
- `doubao` 的 Recall 很高，但 FP 明显偏多，因此 F1 不如 deepseek / qwen。
- 这说明在 Juliet 上，JLeakDetector 的核心优势在于 **高召回且保持较高 Precision**。

表 B2：FitNesse 上 PMD、SpotBugs、qwen-plus、deepseek-chat、doubao 的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden	平均时间（秒）	平均 token
PMD	3.0	2.0	0.6000	51.00	10.20	N/A	N/A
SpotBugs	5.0	0.0	1.0000	10.00	2.00	N/A	N/A
deepseek-chat	3.5	1.5	0.7000	64.25	12.15	838.54	381,632.8
qwen-plus	3.5	1.5	0.7000	68.25	12.95	465.42	349,796.6
doubao-seed-2-0-mini-260215	3.5	1.5	0.7000	65.33	12.37	1760.16	1,179,680.2

结论（B2）

- 在 FitNesse 上，SpotBugs 是最强的传统基线工具，直接达到 **1.0 recall_known**。
- JLeakDetector 三个模型的平均值相同，说明该项目主要受**配置影响**，而不是模型差异影响。
- 成本方面：
  - `qwen-plus` 最快，token 成本也最低；
  - `doubao` 最慢，且 token 成本最高。
- 因此 FitNesse 更适合作为“配置敏感性”讨论对象，而不是单纯的模型优劣讨论对象。

表 B3：TripleA 上 PMD、SpotBugs、qwen-plus、deepseek-chat、doubao 的比较

方法	TP_known	FN	recall_known	候选结果数	inspection_burden	平均时间（秒）	平均 token
PMD	12.0	2.0	0.8571	124.00	8.86	N/A	N/A
SpotBugs	7.0	7.0	0.5000	26.00	1.86	N/A	N/A
deepseek-chat	13.0	1.0	0.9286	88.92	5.17	921.27	637,590.2
qwen-plus	13.0	1.0	0.9286	98.42	5.83	853.37	765,097.2
doubao-seed-2-0-mini-260215	13.0	1.0	0.9286	94.42	5.56	1550.02	1,845,934.2

结论（B3）

- 在 TripleA 上，三种 JLeakDetector 模型的平均 recall_known 都高于 PMD，也显著高于 SpotBugs。
- PMD 的覆盖较高，但候选结果数太多，人工审核负担较重。
- SpotBugs 候选较少，但 known-bug 覆盖明显不足。
- 三个模型里：
  - `qwen-plus` 最快；
  - `doubao` 最贵；
  - `deepseek-chat` 在效果与成本之间更均衡。

表组 C：模块消融实验

> 说明：当前三个模型在消融趋势上高度一致，因此这里使用 `deepseek-chat` 作为代表进行展示。

> soot模块：一个路径预过滤模块。它利用 Soot 生成的静态语义事实，对候选资源泄露路径进行前置检查，在进入 LLM 路径验证之前剪除一部分明显不成立或证据不足的路径。其主要作用是：减少需要送入 LLM 的候选路径数量、降低时间与 token 成本以及在部分项目上抑制噪声报告。
>
> issue-first模块：所谓 issue级审计视角，是指从人工代码审计的角度出发，将多个语义上相关的原始检测证据（如同一资源链上的多个 source、同一问题的多条传播路径、同一位置的重复报告）聚合为一个候选问题单元，以更真实地反映人工审核时面对的候选缺陷列表。

表 C1：FitNesse 上 JLeakDetector 消融结果

变体	recall_known	issue_count	inspection_burden	平均时间（秒）	平均 token
baseline	0.4000	60.00	11.60	898.88	350,484.3
no_issue_first	0.4000	60.67	11.73	763.44	403,441.3
no_soot	1.0000	67.67	12.53	772.94	374,067.3
no_soot_no_issue_first	1.0000	68.67	12.73	918.91	398,538.3

结论（C1）

- FitNesse 上最关键的影响因素是 **soot**。
- 关闭 soot 后，recall_known 从 0.4 直接提升到 1.0,但可以节省token成本。
- `issue-first` 单独关闭并不能带来实质改善，但可以节约时间成本。
- 因此可以认为：**当前soot和issue-first可以压缩一定的token和时间成本，代价是工具的审查负担加重以及可能的召回降低。**

表 C2：TripleA 上 JLeakDetector 消融结果

变体	recall_known	issue_count	inspection_burden	平均时间（秒）	平均 token
baseline	0.8571	88.33	5.21	1019.63	604,449.3
no_issue_first	0.9286	86.00	5.00	841.84	673,212.7
no_soot	0.9286	91.33	5.31	971.97	601,459.3
no_soot_no_issue_first	1.0000	90.00	5.14	851.64	671,239.7

结论（C2）

- TripleA 上 soot 和 issue-first 都会带来一定损失。
- 同时关闭二者时效果最好。
- 与 FitNesse 不同，TripleA 上并不存在 soot 的单因素压倒性负效应，而是 soot 与 issue-first 都需要重新评估。
- 和 FitNesse 类似的是，soot和issue-first可以压缩时间和token成本。
- 这说明当前系统的核心瓶颈之一在于：**模块配置策略与项目特性之间并不匹配，当前的模型配置策略更多的是出于对工具时间和token成本与漏洞检出效果之间的权衡。**

总体总结

1. 与传统工具相比

- 在 **Juliet** 上，JLeakDetector 无论最好还是最坏结果，都显著优于 PMD 与 SpotBugs。
- 在 **真实项目** 上：

    - FitNesse：SpotBugs 最强，JLeakDetector 最佳配置可追平；
    - TripleA：JLeakDetector 明显优于 PMD 和 SpotBugs。

2. 模型比较

- `qwen-plus` 是当前**综合性价比最优模型**；
- `deepseek-chat` 在效果和成本之间较均衡；
- `doubao` 的成本显著偏高，且 Juliet 上 FP 较多，不适合作为当前主力模型。

3. 模块作用

- 当前模块配置（尤其 soot 和 issue-first）对结果的影响大于模型差异本身；
- FitNesse 上 soot 明显有负效应；
- TripleA 上 soot 和 issue-first 同时关闭效果最好。
- soot 和 issue-first 可以降低时间和token成本，这两个模块的存在属于工具对时间和token成本与漏洞检出效果之间的权衡。

4. 系统定位

- 当前系统已经表现出较强的 benchmark 覆盖能力和真实项目已知 bug 发现能力；
- 但在某些项目上 `inspection_burden` 仍然偏高，不能做到在所有项目上都将`inspection_burden`控制在5左右，说明它更适合作为**高召回、辅助式的资源泄露审计工具**，而非低噪声、生产级自动告警工具。

调用深度参数敏感性补充实验（qwen-plus）

> 说明：下面两张表基于 qwen-plus 在 `call_depth = 5 / 10 / 15` 下的真实项目实验结果整理。
> 每个表中的指标均为对应项目下 **4 个变体 × 3 次重复** 的平均值。

表 D1：FitNesse 上 `call_depth = 5 / 10 / 15` 的对比（qwen-plus）

call_depth	recall_known	Top-50% Recall	MRR	issue_count	inspection_burden	平均时间（秒）	平均 token
5	0.7000	0.5500	0.0804	67.42	12.78	458.69	348,728.8
10	0.7000	0.5167	0.0441	68.08	12.92	**422.87**	**343,120.5**
15	0.7000	0.5333	0.0455	68.25	12.95	465.42	349,796.6

结论（D1）

- 三种调用深度的 `recall_known` **完全一致**，说明更深的跨过程传播并没有带来额外覆盖收益。
- `call_depth=5` 的排序质量最好：
    - `Top-50% Recall` 最高；
    - `MRR` 最高。
- `call_depth=10` 的成本最低：
    - 平均时间最短；
    - 平均 token 最少。
- 综合来看：
    - 若优先考虑**效率**，`call_depth=10` 更合适；
    - 若优先考虑**排序质量**，`call_depth=5` 略有优势。

表 D2：TripleA 上 `call_depth = 5 / 10 / 15` 的对比（qwen-plus）

call_depth	recall_known	Top-50% Recall	MRR	issue_count	inspection_burden	平均时间（秒）	平均 token
5	0.9286	0.3929	0.0352	98.58	5.84	860.68	768,405.5
10	**0.9345**	0.3869	0.0355	**97.17**	**5.73**	867.75	**762,342.0**
15	0.9286	**0.3929**	**0.0388**	98.42	5.83	**853.37**	765,097.2

结论（D2）

- 三种调用深度的整体效果差异很小。
- `call_depth=10` 的 `recall_known` 略高，且：
  - `issue_count` 最低；
  - `inspection_burden` 最低；
  - token 成本最低。
- `call_depth=15` 的 `MRR` 最高，说明更深传播在排序上可能仍有轻微优势。
- `call_depth=5` 并未体现出明显收益。
- 综合来看：
  - TripleA 对调用深度**不敏感**；
  - `call_depth=10` 是更均衡的折中选择。

对于调用深度对实验影响的结论：

从两个项目实验表现来看，调用深度对实验效果的影响并不是线性单调的，而是取决于项目的规模和特性。

并发度参数敏感性补充实验（qwen-plus）

> 说明：下面两张表基于 qwen-plus 在 `max_neural_workers = 16 / 8 / 4` 下的真实项目实验结果整理。
> 每个表中的指标均为对应项目下 **4 个变体 × 3 次重复** 的平均值。

表 S3：FitNesse 上 `max_neural_workers = 16 / 8 / 4` 的对比（qwen-plus）

并发数	recall_known	Top-50% Recall	MRR	issue_count	inspection_burden	平均时间（秒）	平均 token	平均 query_count
16	0.7000	0.5333	0.0455	68.25	12.95	**465.42**	**349,796.6**	281.0
8	0.7000	**0.5500**	**0.0629**	68.92	13.08	608.41	365,429.2	293.3
4	0.7000	0.5167	0.0465	68.42	12.98	940.86	360,314.8	286.5

结论（S3）

- 三种并发度下 `recall_known` 完全一致，说明降低并发不会改变已知缺陷覆盖率。
- 并发数为 8 时，排序指标略好：
  - `Top-50% Recall` 最高；
  - `MRR` 最高。

- 但从成本看：
  - 并发 16 最快；
  - 并发 16 的 token 也最低；
  - 并发 4 最慢。
- 因此，对 FitNesse 而言，降低并发主要带来**时间损失**，而收益仅体现在非常有限的排序改善上。

表 S4：TripleA 上 `max_neural_workers = 16 / 8 / 4` 的对比（qwen-plus）

并发数	recall_known	Top-50% Recall	MRR	issue_count	inspection_burden	平均时间（秒）	平均 token	平均 query_count
16	0.9286	**0.3929**	**0.0388**	98.42	**5.83**	**853.37**	765,097.2	472.3
8	0.9286	0.3690	0.0262	99.17	5.89	1357.27	**763,789.0**	467.8
4	0.9345	0.3452	0.0344	99.17	5.88	2086.54	766,952.4	467.3

结论（S4）

- TripleA 上并发数变化对 `recall_known` 的影响很小，三种并发配置差异不显著。
- 但排序指标整体上以并发 16 最好：
  - `Top-50% Recall` 最高；
  - `MRR` 最高。
- 并发数降低后，时间显著增加：
  - 并发 8 明显更慢；
  - 并发 4 极慢。
- token 总量变化很小，说明并发数主要影响的是**运行时间**而不是**总 token 消耗**。

并发度变化的总体结论

- 对于 qwen-plus 而言，将 `max_neural_workers` 从 16 降低到 8 或 4，并不会显著改变已知 bug 覆盖率。
- 在 FitNesse 上，较低并发有时会带来轻微的排序改善，但不足以抵消时间成本上升。
- 在 TripleA 上，较低并发既没有明显提升覆盖率，反而显著拉长了总运行时间，并削弱了部分排序指标。
- 因此，在当前实现下，`max_neural_workers = 16` **仍然是更合理的默认设置**；并发数下降带来的主要效果是效率变差，而非检测质量提升。